The `IOptions` interface is canonical in ASP.NET Core; it's used by the core ASP.NET Core libraries and has various convenience functions for binding strongly typed settings, as you've already seen. In many cases, however, the `IOptions` interface doesn't give many benefits for *consumers* of the strongly typed settings objects. Services must take a dependency on the `IOptions` interface but then immediately extract the real object by calling `IOptions<T>.Value`. This situation can be especially annoying if you're building a reusable library that isn't inherently tied to ASP.NET Core, as you must expose the `IOptions<T>` interface in all your public APIs.

Luckily, the configuration binder that maps `IConfiguration` objects to strongly typed settings objects isn't inherently tied to `IOptions`. Listing 10.11 shows how you can bind a strongly typed settings object to a configuration section manually, register it with the DI container, and inject the `MapSettings` object directly into a handler or service without the additional ceremony required to use `IOptions<MapSettings>`.

Listing 10.11 Configuring strongly typed settings without `IOptions` in Program.cs

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

var settings = new MapSettings ();                                    ❶
builder.Configuration.GetSection("MapSettings").Bind(settings);       ❷
builder.Services.AddSingleton(settings);                              ❸

WebApplication app = builder.Build();

app.MapGet("/", (MapSettings mapSettings) => mapSettings);            ❹

app.Run();
```

❶ Creates a new instance of the MapSettings object

❷ Binds the MapSettings section in IConfiguration to the settings object

❸ Registers the settings object as a singleton

❹ Injects the MapSettings object directly using DI